

Comprehensive Q&A; Guide for C-Based Round Robin Load Balancer Project

1. High-Level & Conceptual Questions

Q1. What is the main goal of your project?

A: To implement a load balancer that distributes client connections evenly among backend proxies using round-robin scheduling.

Q2. What is load balancing and why is it important?

A: Load balancing distributes workload across servers to improve reliability, availability, and performance.

Q3. What algorithm did you use?

A: Round Robin — each request goes to the next proxy cyclically.

Q4. Why Round Robin?

A: It's simple, fair, and efficient for equally capable servers.

Q5. Layer 4 or Layer 7?

A: It's a Layer 4 TCP load balancer, forwarding raw TCP streams.

2. Architecture & Design

Q6. Architecture?

A: The balancer listens on a public port, selects a proxy via round-robin, and spawns a thread to forward traffic between client and proxy using `select()`.

Q7. Thread safety?

A: Protected with pthread mutex during index rotation.

Q8. Data structure?

A: Array of proxy structs {ip, port} with round-robin index.

3. Networking & Implementation

Q9. Socket functions used?

A: socket(), bind(), listen(), accept(), connect(), select(), send(), recv().

Q10. Why select()?

A: Portable and efficient for moderate connections.

Q11. Handling proxy down?

A: Logs error; can extend with health checks.

Q12. Bidirectional communication?

A: select() monitors both sockets, forwarding data both ways.

4. Performance & Optimization

Q13. Proxy selection complexity?

A: $O(1)$ using modulo arithmetic.

Q14. Scalability?

A: Good up to hundreds of clients; can improve using epoll or thread pools.

Q15. Latency?

A: Minimal — direct TCP forwarding without buffering.

5. Reliability & Error Handling

Q16. Error handling?

A: Checks recv()/send() results, closes sockets gracefully.

Q17. File descriptor management?

A: Proper cleanup and use of SO_REUSEADDR.

6. Advanced Extensions

Q18. Adding health checks?

A: Periodically ping proxies, mark inactive if no response.

Q19. Least-connections balancing?

A: Track active connection count per proxy and pick lowest.

Q20. HTTPS support?

A: Works transparently at Layer 4; TLS termination needs OpenSSL.

7. Code-Specific

Q21. pthread_detach vs join?

A: detach() cleans threads automatically.

Q22. Why SO_REUSEADDR?

A: Allows immediate rebinding after restart.

Q23. Mutex usage?

A: To safely update shared round-robin index.

8. Open-Ended & Discussion

Q24. Real-world use?

A: Useful for web, proxy, or caching server clusters.

Q25. Learnings?

A: Socket programming, threading, synchronization, and I/O multiplexing.

Q26. Future improvements?

A: Add health checks, switch to epoll, config hot-reload, Dockerize.

Q27. Efficiency factors?

A: Constant-time selection, minimal buffering, lightweight threading.

Q28. Unique value?

A: Lightweight, transparent, and educational vs heavy tools like HAProxy.